RESEARCH IMPLEMENTATION / EVIDENCE RECORD

Diabetic retinopathy grading with Vision Mamba

Architecture, data controls, verification and delivery status

Dharan Tej Reddy 4 November 2026

Scope of this document. This implementation dossier records the code and preparation work. It is **not** the assignment's final experimental report. A full training run, held-out classification metrics, prediction figures and a result-bearing LaTeX report do not yet exist. Two recorded optimizer steps are qualification checks only.

Project links

GitHub source repository · Vision Mamba paper · Original EyePACS release

Source code and documentation are public. Retinal images, labels, credentials, private access links and diagnostic artifacts remain in private storage.

1 Assignment and delivery state

The internship brief asks for a Vision Mamba classifier implemented from scratch in PyTorch, a custom EyePACS `Dataset` used with `DataLoader`, five-grade training, held-out EyePACS and Messidor evaluation, metrics and example predictions, and a compiled LaTeX results report. It evaluates code quality, architecture understanding, analysis, documentation and reproducibility. The last two output groups still depend on a completed experiment.

Expected deliverable	Existing implementation	Evidence state
PyTorch Vim	Own model blocks, reference and parallel scan, five-logit head	Source exists; earlier revision passed two real-image optimizer steps
Dataset/preprocessing	Own retinal `Dataset`; full-field transform; manifest, split and audit; PyTorch `DataLoader`	35,126 original EyePACS images audited privately
Train/evaluate scripts	CLI, qualification, checkpointed training, sealed evaluation, metrics and report generator	Code tested; full epoch and sealed evaluation unrun
Metrics and predictions	Result generators reject incomplete runs	No measured test results
Compiled results report	LaTeX generator and compiler smoke check	No final experiment PDF

For a code-focused review, the first three deliverables are available as source. That is a reduced-scope submission against the PDF, because the PDF also asks for real training and results.

2 Architecture and paper mapping

The Vision Mamba paper represents images as sequences of position-marked patches and uses selective state-space processing in both directions to build global context [1]. This implementation uses a middle class token and independent forward/reverse convolution and SSM parameters. It does not import a ready-made Vim model or pretrained weights. The authors' implementation informed the architecture review, not the model dependency.

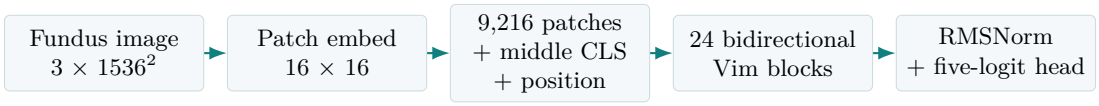


Figure 1. Implemented inference path. These are configured dimensions, not a measured clinical operating point.

For image side $H = 1536$, patch side $P = 16$, width $D = 768$ and five grades, the token count and shapes are

$$N = (H/P)^2 = 96^2 = 9{,}216, \qquad L = N + 1 = 9{,}217,$$

$$\text{patch tokens} \in \mathbb{R}^{B\times N\times 768}, \qquad \text{class logits} \in \mathbb{R}^{B\times 5}.$$

The configured block uses expansion factor two, directional width $E = 1{,}536$, state dimension $S = 16$, causal convolution width four, and depth 24. The instantiated classifier has 103,760,645 trainable parameters. The 1536-pixel input and 300-epoch schedule are project choices, not settings mandated by the paper or assignment brief.

3 Bidirectional selective state-space block

Every block RMS-normalizes its input, projects features and a shared gate, and processes the features forward and in reverse token order. Each direction has its own causal depthwise convolution, input-dependent state projections, step-size projection, negative diagonal state matrix and skip coefficient. The outputs are returned to the original order, summed, gated, projected and added to an FP32 residual.

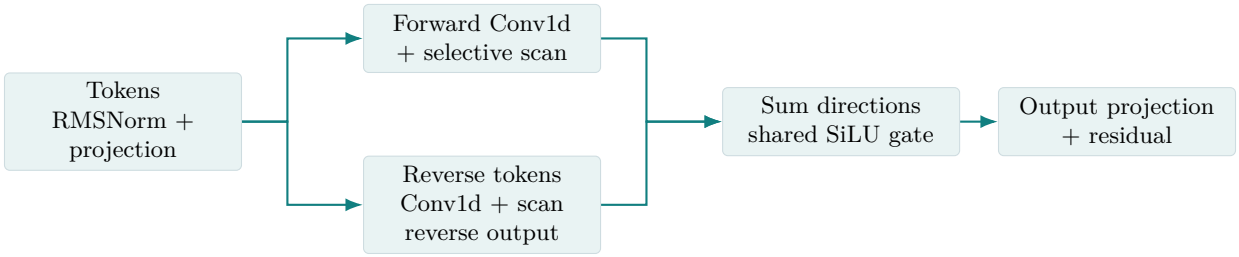


Figure 2. The forward and reverse SSM/convolution parameters are independent; the gate is shared.

In the reference scan, Δ_t and B_t, C_t depend on current input, $A = -\exp(A_{\log})$, and the state update uses FP32:

$$\bar{A}_t = \exp(\Delta_t A), \qquad h_t = \bar{A}_t \odot h_{t-1} + \Delta_t B_t \odot x_t, \qquad y_t = \langle C_t, h_t \rangle + D_{\text{skip}} \odot x_t.$$

The input drive follows the released Mamba/Vim selective-scan convention $\Delta_t B_t$ [2]. The primary parallel backend composes affine transitions in bounded chunks and implements its own backward recurrence. It does not reproduce the paper's fused SRAM kernel, so the paper's published speedups are not claims about this implementation.

Path	Purpose	Present evidence
Reference	Transparent recurrence and gradient comparison	Mathematical source; local parity tests
Parallel	Primary from-scratch PyTorch training path	Earlier BF16 source passed real-image parity and two model steps
Pinned CUDA	Optional low-level scan efficiency path, never a ready-made Vim model	Scan-only benchmark passed; current whole model unqualified

Implementation map

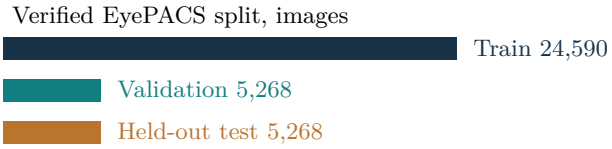
`src/drvim/model/vim.py` defines the classifier, blocks and directional mixers. `src/drvim/model/scan.py` defines the reference, in-house parallel and optional pinned kernel paths. `src/drvim/qualify_scan.py` compares numerics and runs real-image optimizer preflight before training can accept a qualification artifact.

4 Data architecture and leakage controls

The original EyePACS training release has 35,126 labeled JPEGs. The private preparation job read all five official archive parts, checked archive CRCs, decoded and SHA-256 hashed every image, reconciled it to a label, and assigned a grade-stratified split that keeps filename-paired left and right eyes together.



Figure 3. Preparation and test boundary. Filename pairing tokens do not independently prove unique patient identity.



The active custom `RetinaDataset` loads records for its declared source and split. Its transform applies EXIF orientation, converts to RGB, rescales without cropping the retinal field, pads to a square, and allows horizontal flipping only for training. PyTorch's `DataLoader` supplies workers and batching; no ready-made retinal dataset loader is used.

A separate high-assurance candidate adds pinned manifest identity, per-read byte hashing, size and format limits, ICC-to-sRGB conversion, deterministic augmentation, and metadata evidence gates. It is **not connected to the primary training run**. The public EyePACS release lacks verified patient linkage, site, camera, acquisition date, adjudication and rights metadata; the candidate mode therefore fails closed instead of inventing those fields.

Concern	Active assignment loader	Optional high-assurance candidate
Source identity	Manifest checked before run	Manifest pinned and image bytes rehashed at sample read
Color	RGB conversion	Embedded ICC to sRGB; untagged assumption recorded
Geometry	Full-field letterbox	Same geometry plus occupancy metadata
Augmentation	Train-only random flip	Seed/epoch/image-ID deterministic flip
Clinical metadata	Unavailable	Explicit patient/site/device/grade/rights attestations required

Source map for data handling

File	Responsibility
`data/prepare.py`, `data/manifest.py`	Official cohort adapters, labels, hashes and strict manifest schema
`data/split.py`, `data/audit.py`	Grouped assignment and leakage/source checks
`data/dataset.py`, `data/preprocess.py`	Active custom Dataset and full-field image transform
`data/high_assurance_dataset.py`	Separate candidate for stronger provenance and image controls

Official original Messidor preparation

All 12 archives and 12 canonical annotation workbooks passed private integrity checks. The preparer decoded and SHA-256 hashed 1,200 TIFFs, applied four published grade corrections, and excluded 13 published Base33 duplicates. Its schema-valid manifest has 1,187 eligible external images: R0 547, R1 149, R2 240 and R3 251. Included image SHA-256 sets have zero exact overlap with the verified EyePACS manifest; near duplicates are not excluded by this check. No external model predictions exist.

5 Training and external evaluation

The classifier is randomly initialized. The training code defines cross-entropy, AdamW with parameter-specific decay exclusions, warmup and cosine learning rate, mixed precision, gradient clipping, accumulation, activation checkpointing and distributed data parallel execution. Each completed epoch saves model, optimizer, scheduler, scaler, RNG, config and manifest identity. Highest EyePACS validation QWK selects the checkpoint. The held-out EyePACS and Messidor tests never select hyperparameters.

$$\mathcal{L}_{\mathrm{CE}} = -\frac{1}{M}\sum_{i=1}^{M} \log p_\theta(y_i \mid x_i), \qquad y_i \in \{0,1,2,3,4\}.$$

Distributed padding examples have zero contribution so each real training image contributes once to the global loss. An epoch-boundary pause saves a checkpoint without writing the full-run completion marker.

Declared configuration (not a completed run)

Setting	Value	Setting	Value
Epochs / seed	300 / 2026	Batch / accum.	2 / 4 per GPU
AdamW learning rate	10^{-4}	Decay / warmup	0.05 / 5 epochs
Precision	FP16 AMP	Clipping / drop path	1.0 / 0.1
Workers per rank	8	Selection criterion	Validation QWK

Original Messidor is the external dataset named by the brief. Its R0–R3 grades differ from EyePACS ICDR 0–4; five-class QWK against original Messidor would be invalid. The declared cross-source binary proxy is Messidor R2/R3 versus model ICDR 2/3/4. Optional Messidor-2 uses separately published adjudicated grades 0–4 [4, 5]. Neither external cohort enters training or checkpoint selection. ADCIS states that 1,058 Messidor-2 PNGs come from original Messidor and 690 JPGs form an extension; the two named cohorts are therefore not independent replications.

The evaluation command requires a completed-run marker, pins the source manifest and selected checkpoint, verifies images, then records per-image probabilities and predictions. It calculates accuracy, precision, recall, F1, QWK where valid, confusion matrices and loss curves. The report generator recomputes metrics from saved rows before placing plots and image pairs in LaTeX. **These generators exist; no classification results have been generated yet.**

Declared metric definitions

For a five-grade cohort, the confusion matrix contains integer counts $C_{ij} = \#\{n : y_n = i, \hat{y}_n = j\}$. Accuracy is the fraction of correct predictions; class precision, recall and F1 are computed with explicit support. Macro values average the five class values equally. Quadratic weighted kappa is

$$\kappa_{\mathrm{QW}} = 1 - \frac{\sum_{i,j} w_{ij} O_{ij}}{\sum_{i,j} w_{ij} E_{ij}}, \qquad w_{ij} = \frac{(i-j)^2}{(K-1)^2}, \quad K = 5,$$

where O is observed grade-pair frequency and E is the expected frequency from the observed marginals. Undefined QWK is rejected rather than replaced with a fabricated zero. Original Messidor receives its declared binary endpoint only; five-grade QWK is reserved for commensurate ICDR labels.

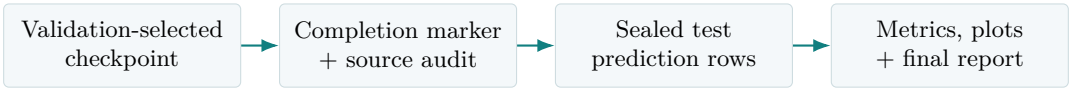


Figure 5. Implemented result gate. The rightmost artifacts cannot exist honestly until the preceding experimental stages complete.

6 What has actually run

Check	Observed result	Interpretation
Original EyePACS preparation	35,126 labeled images decoded and hashed; all five grades in each split; no filename-pair group crosses splits	Complete data audit, not model training
Earlier BF16/parallel qualification	One Tesla T4, 1536-pixel input, 103,760,645 parameters; two eight-image optimizer steps of 195.71 and 197.24 seconds; peak allocation 4.64 and 5.47 GB	Two-step diagnostic only
Scan-only comparison	At 9,217 tokens on T4, in-house FP32 scan 0.334 s versus pinned CUDA scan 0.0256 s; forward maximum absolute difference 1.31×10^{-6}	About 13 times faster for this scan benchmark only
Repository checks	Ruff passed and 12 local tests passed	Source checks, not accuracy

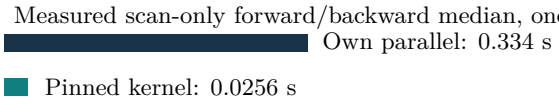


Figure 4. This compares only a selective-scan direction, not a complete model step, epoch or classifier.

The kernel timing does not establish full-model throughput. The two-step qualification is pinned to an earlier BF16/parallel source revision. The current FP16/optional CUDA code requires its own exact-source qualification. At the earlier T4 rate, the configured 300-epoch experiment cannot fit the available free Kaggle time budget. The brief does not require 300 epochs. No paid environment has been used.

7 Stack and reproducibility

Technology	Role and boundary
Python 3.10+	Package, CLI, manifests and acquisition verification
PyTorch	Tensors, autodiff, `Dataset`/`DataLoader`, AMP, DDP and layers; model and training loop are authored in this repository
NumPy / pandas	Array conversion, annotation tables and manifests
Pillow	JPEG/TIFF decode, EXIF correction, resize and optional color control
scikit-learn	Metrics and confusion counts, not a ready-made retinal model
Matplotlib	Plots only after actual evaluation
PyYAML	Recorded experiment configuration
xlrd / openpyxl	Official Messidor workbook parsing
Tectonic / pdfLaTeX	Result report compilation
Kaggle	Private EyePACS input and GPU diagnostic jobs
Git / GitHub	Public source and docs; no images, credentials or weights

The public repository is github.com/dharan1007/internship-task-1. The audited EyePACS split manifest SHA-256 is `f933f1bcff8f99688fd8a44984acaa4ce0c929d899be9bddcabdfc54ea7c2117`. The hash identifies the exact split; raw licensed images remain private. Source, manifest, config, checkpoint and report artifact identities are recorded so a finished experiment can be traced back to its inputs.

8 Access and support timeline

Date	Recorded event
3 November 2026	Requested exact-scope EyePACS internship use and image-in-report permission. Submitted ADCIS Messidor and Messidor-2 access forms and contacted ADCIS support when personal links did not arrive.
4 November 2026	EyePACS contact directed use of its Kaggle release and requested EyePACS, Inc. and Cuadros–Bresnick (2009) credit. This did not expressly settle every proposed training, weight-sharing and image-report scope.
4 November 2026	Original EyePACS audit and grouped split completed in private storage. Earlier model two-step qualification and separate CUDA scan comparison completed on Kaggle.
4 November 2026	ADCIS technical support replied with restricted alternative access for original Messidor and Messidor-2 after the form issue. Private links and credentials are omitted here.
4 November 2026	All 12 original Messidor archives passed private size, hash and ZIP checks. The 1,200 TIFFs and 12 canonical annotation workbooks yielded a schema-valid manifest with 1,187 eligible images after 13 published duplicate exclusions.
4 November 2026	Four official Messidor-2 image parts passed byte-size and hash checks. The assembled ZIP passed CRC; all 1,748 extracted images passed full decode and byte-hash uniqueness checks. Official pairing is unresolved.
4 November 2026	Compared the block with Vim Algorithm 1 and corrected directional combination to a sum before the shared gate and projection. Earlier GPU qualification does not certify this revised source.

The official Messidor and Messidor-2 pages permit free research and educational use while prohibiting copying and redistribution. They request acknowledgment of the Messidor program ("Kindly provided by the Messidor program partners") and, for Messidor-2, the LaTIM laboratory [4, 5, 6]. Restricted access details and images are not in this dossier or the public repository.

9 Kaggle evidence and source links

1. Prepare Original EyePACS: official-source preparation, 35,126 JPEGs decoded and hashed, label reconciliation and fixed split. Produced the manifest used for subsequent qualification.
2. Full Model Qualification: checked FP32/BF16 scan parity and executed two complete optimizer steps on real EyePACS images under the pinned full-split identity. It is not an epoch or validation measurement.
3. Pinned CUDA Scan Build: built a pinned low-level selective-scan wheel in the Kaggle environment, with no pretrained weights.
4. Real EyePACS CUDA Scan Comparison: compared the in-house scan and the low-level kernel on real-image-derived inputs at intended token length. Reports kernel-level timing and numerical differences, not classifier accuracy or whole-model runtime.

These notebooks may require owner access. None is a live full training job.

10 Remaining work and limits

1. Run the exact-source and cross-cohort audit with the completed original Messidor manifest before sealed inference. The verified images and manifest alone are not external model results.
2. Requalify the current FP16 and optional CUDA full-model source, measure a complete epoch and distributed throughput, and find a zero-cost allocation that can finish a predeclared training schedule.
3. Train five-grade classification fully, selecting the checkpoint on EyePACS validation only. Preserve checkpoint and run identity.
4. Run sealed held-out EyePACS and original Messidor evaluation; generate measured metrics, plots and real example predictions. Add Messidor-2 only after its official images and pairing metadata are reconciled.
5. Compile and inspect the final *results* LaTeX report. Confirm applicable EyePACS permission before sharing image-bearing PDFs or weights.

This is research software, not a medical device or clinical diagnostic claim. Public EyePACS filenames do not independently prove patient identity. Acquisition, quality, grading-standard and population differences may affect external generalization.

References

[1] L. Zhu et al., "Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model," ICML 2024. https://arxiv.org/abs/2401.09417.

[2] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," 2023. https://arxiv.org/abs/2312.00752.

[3] J. Cuadros and G. Bresnick, "EyePACS: An Adaptable Telemedicine System for Diabetic Retinopathy Screening," *Journal of Diabetes Science and Technology*, 3(3):509–516, 2009. EyePACS, Inc. is credited for the original https://www.kaggle.com/competitions/diabetic-retinopathy-detection/data retinal image release.

[4] E. Decencière et al., "Feedback on a publicly distributed database: the Messidor database," *Image Analysis & Stereology*, 33(3):231–234, 2014. Kindly provided by the Messidor program partners. https://www.adcis.net/en/third-party/messidor/.

[5] Messidor program partners, "Messidor-2" access and acknowledgment. https://www.adcis.net/en/third-party/messidor2/.

[6] M. D. Abràmoff et al., "Automated analysis of retinal images for detection of referable diabetic retinopathy," *JAMA Ophthalmology*, 131(3):351–357, 2013. https://doi.org/10.1001/jamaophthalmol.2013.1743.

[7] Google Brain, "Messidor-2 DR Grades." https://www.kaggle.com/datasets/google-brain/messidor2-dr-grades.